ETL Pipeline using PostgreSQL

ON GROCERY SUPERMARKET TRANSACTION DATA

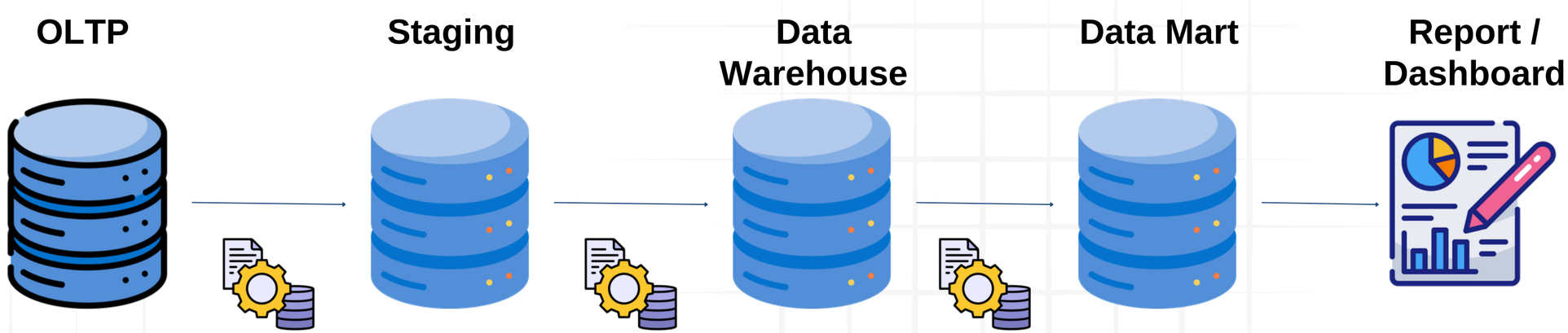
by : Muhammad Rafly Alviansyah

Project Background & Objectives

Mengolah data mentah transaksi supermarket periode Januari - Mei dari skema **Staging**, ditransformasikan ke skema **Data Warehouse** (DWH), hingga membentuk ringkasan siap pakai di **Data Mart** (DM).

- Membangun data pipeline yang otomatis, terstruktur, dan bebas dari risiko redundansi data.
- Menganalisis tren transaksi bisnis harian berdasarkan waktu (per jam).
- Menganalisis Product yang terjual
- Menganalisis performa tiap Store di berbagai wilayah

Data Pipeline



1. Phase OLTP ➡️ Staging (**Data Ingestion**): Fokus pada kecepatan transfer raw data ke database tanpa transformasi berat.

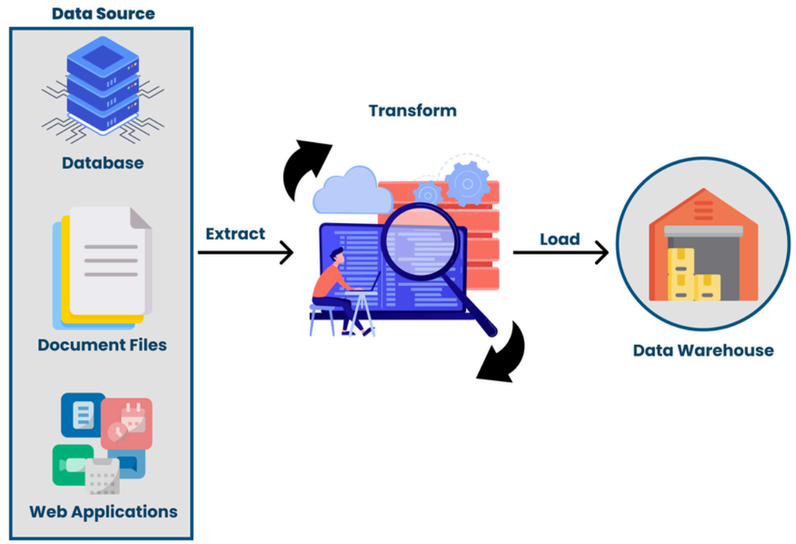
2. Phase Staging ➡️ DWH (**Cleansing & Modeling**): Raw data dipecah menjadi **Star Schema** yang memisahkan **fact table** dan **dimensi** (dim_product, dim_store, dim_customer).

3. Phase DWH ➡️ DM (**Aggregation** & **Automation**): Data detail diringkas menjadi tabel agregasi bisnis yang siap pakai lewat bantuan Stored Procedure.

4. Phase Data Mart ➡️ Report / Dashboard (**Data Consumption**): Proses konsumsi data oleh Power BI untuk kebutuhan visualisasi interaktif.

Apa itu ETL?, Mengapa Penting?

ETL **(Extract, Transform, Load)** adalah automated data pipeline yang berfungsi untuk mengekstrak raw data dari sistem operasional, membersihkan dan memodelkan strukturnya, lalu memuatnya ke dalam repositori analitik terpusat agar siap dikonsumsi secara instan oleh BI Tools.



Data Consistency:

Menjamin format data di seluruh cabang (Jakarta, Bandung, Bogor, dll) terstandardisasi sama rata.

Single Source of Truth:

Seluruh laporan internal manajemen (keuangan, logistik, marketing) mengacu pada basis data terpusat yang sama.

Performance Isolation

Memisahkan database operasional toko dengan database laporan agar penarikan grafik visual tidak mengganggu sistem mesin kasir di lapangan.

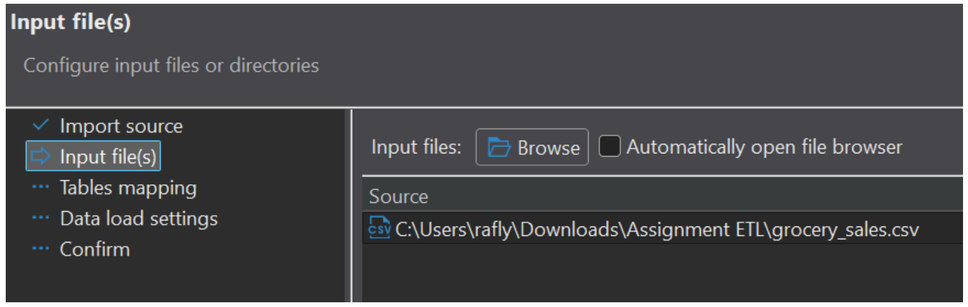
Staging Area

```sql
-- ============================================
-- STEP 1 - STAGING (EXTRACT)
-- ============================================
CREATE SCHEMA IF NOT EXISTS staging;

DROP TABLE IF EXISTS staging.stg_grocery_sales;

CREATE TABLE staging.stg_grocery_sales (
    sales_id            INT,
    transaction_number  TEXT,
    sales_date          TIMESTAMP,
    store_id            INT,
    store_city          TEXT,
    customer_id         INT,
    first_name          TEXT,
    last_name           TEXT,
    product_id          INT,
    product_name        TEXT,
    category_name       TEXT,
    quantity            INT,
    price               NUMERIC(12, 2),
    discount            NUMERIC(10, 2),
    sales               NUMERIC(14, 2)
);
```

Buat Schema Staging

Buat menghindari duplicate

Buat tabel



Data Profiling

```
# — EXPLORE
print(df.dtypes)
print(df.isnull().sum())
```

```
sales_id              int64
transaction_number    object
sales_date            object
store_id               int64
store_city            object
customer_id            int64
first_name            object
last_name             object
product_id             int64
product_name          object
category_name         object
quantity               int64
price                float64
discount             float64
sales                float64
dtype: object
sales_id              0
transaction_number    0
sales_date            0
store_id              0
store_city            0
customer_id           0
first_name            0
last_name             0
product_id            0
product_name          0
category_name         0
quantity              0
price                 0
discount              0
sales                 0
dtype: int64
```

Tidak terdapat null

```
print(df.duplicated().sum())
```

```
0
```

Tidak terdapat duplicate juga

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 59424 entries, 0 to 59423
Data columns (total 15 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   sales_id            59424 non-null  int64
 1   transaction_number  59424 non-null  object
 2   sales_date          59424 non-null  object
 3   store_id            59424 non-null  int64
 4   store_city          59424 non-null  object
 5   customer_id         59424 non-null  int64
 6   first_name          59424 non-null  object
 7   last_name           59424 non-null  object
 8   product_id          59424 non-null  int64
 9   product_name        59424 non-null  object
 10  category_name       59424 non-null  object
 11  quantity            59424 non-null  int64
 12  price               59424 non-null  float64
 13  discount            59424 non-null  float64
 14  sales               59424 non-null  float64
dtypes: float64(3), int64(5), object(7)
memory usage: 6.8+ MB
```

Data Warehouse (dimension)

```sql
-- ============================================================
-- STEP 2 - DIMENSI (TRANSFORM)
-- Ekstrak data unik dari staging ke masing-masing dimensi
-- ============================================================
CREATE SCHEMA IF NOT EXISTS dwh;
```

```sql
-- ----------------------------------
-- DIM CUSTOMER
-- (ditambah kolom full_name sebagai kolom turunan)
-- ----------------------------------
DROP TABLE IF EXISTS dwh.dim_customer;

CREATE TABLE dwh.dim_customer (
    customer_id INT PRIMARY KEY,
    first_name  TEXT,
    last_name   TEXT,
    full_name   TEXT
);

INSERT INTO dwh.dim_customer (customer_id, first_name, last_name, full_name)
SELECT DISTINCT
    customer_id,
    first_name,
    last_name,
    first_name || ' ' || last_name
FROM staging.stg_grocery_sales
ORDER BY customer_id;
```

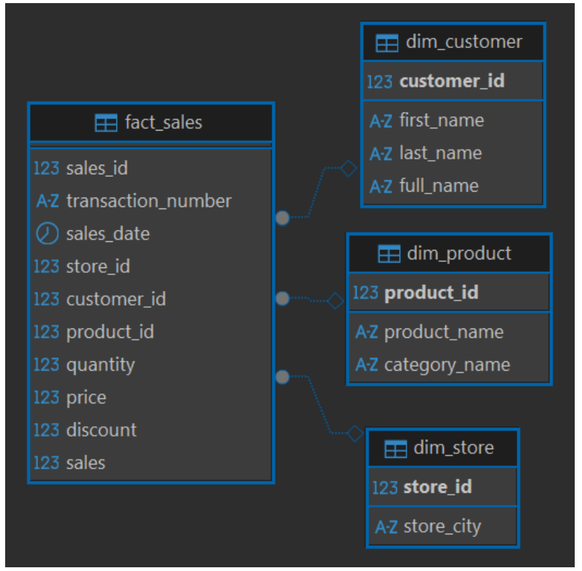
Insert data dari staging

```sql
-- ----------------------------------
-- DIM PRODUCT
-- ----------------------------------
DROP TABLE IF EXISTS dwh.dim_product;

CREATE TABLE dwh.dim_product (
    product_id   INT PRIMARY KEY,
    product_name TEXT,
    category_name TEXT
);

INSERT INTO dwh.dim_product (product_id, product_name, category_name)
SELECT DISTINCT
    product_id,
    product_name,
    category_name
FROM staging.stg_grocery_sales
ORDER BY product_id;
```

```sql
-- ----------------------------------
-- DIM STORE
-- ----------------------------------
DROP TABLE IF EXISTS dwh.dim_store;

CREATE TABLE dwh.dim_store (
    store_id   INT PRIMARY KEY,
    store_city TEXT
);

INSERT INTO dwh.dim_store (store_id, store_city)
SELECT DISTINCT store_id, store_city
FROM staging.stg_grocery_sales
ORDER BY store_id;
```

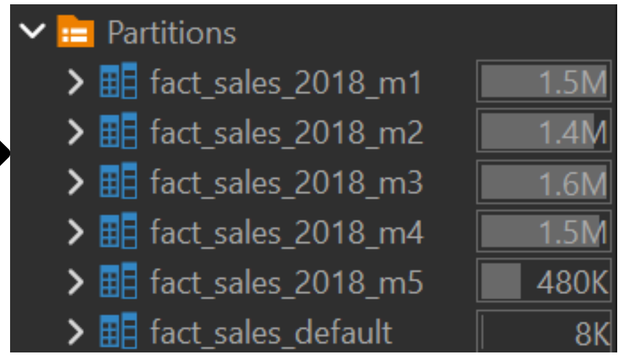
Data Warehouse (fact_table)

```sql
-- ================================================================================
-- STEP 3 - FACT TABLE
-- ================================================================================

DROP TABLE IF EXISTS dwh.fact_sales CASCADE;

CREATE TABLE dwh.fact_sales (
    sales_id           INT,
    transaction_number TEXT,
    sales_date         TIMESTAMP,
    store_id           INT,
    customer_id        INT,
    product_id         INT,
    quantity           INT,
    price              NUMERIC(12, 2),
    discount           NUMERIC(10, 2),
    sales              NUMERIC(14, 2),

    CONSTRAINT fk_sales_store FOREIGN KEY (store_id) REFERENCES dwh.dim_store(store_id),
    CONSTRAINT fk_sales_customer FOREIGN KEY (customer_id) REFERENCES dwh.dim_customer(customer_id),
    CONSTRAINT fk_sales_product FOREIGN KEY (product_id) REFERENCES dwh.dim_product(product_id)
) PARTITION BY RANGE (sales_date);
```



hubungkan dengan table dimensions

```sql
-- BUAT LACI PARTISI PER BULAN TAHUN 2018
CREATE TABLE dwh.fact_sales_2018_m1 PARTITION OF dwh.fact_sales
    FOR VALUES FROM ('2018-01-01 00:00:00') TO ('2018-02-01 00:00:00');

CREATE TABLE dwh.fact_sales_2018_m2 PARTITION OF dwh.fact_sales
    FOR VALUES FROM ('2018-02-01 00:00:00') TO ('2018-03-01 00:00:00');

CREATE TABLE dwh.fact_sales_2018_m3 PARTITION OF dwh.fact_sales
    FOR VALUES FROM ('2018-03-01 00:00:00') TO ('2018-04-01 00:00:00');

CREATE TABLE dwh.fact_sales_2018_m4 PARTITION OF dwh.fact_sales
    FOR VALUES FROM ('2018-04-01 00:00:00') TO ('2018-05-01 00:00:00');

CREATE TABLE dwh.fact_sales_2018_m5 PARTITION OF dwh.fact_sales
    FOR VALUES FROM ('2018-05-01 00:00:00') TO ('2018-06-01 00:00:00');

-- Jaring pengaman cadangan (jika ada data di luar bulan 1-5)
CREATE TABLE dwh.fact_sales_default PARTITION OF dwh.fact_sales DEFAULT;
```



Data Mart (DM)

```sql
-- ============================================
-- STEP 4 - DATAMART VIA STORED PROCEDURE
-- ============================================
CREATE SCHEMA IF NOT EXISTS dm;
```

```sql
-- Tabel A: Ringkasan Per Jam (Brief 1)
CREATE TABLE dm.summary_hourly_metrics (
    sales_hour          INT,
    total_transaksi     INT,
    total_pelanggan     INT,
    total_quantity      INT,
    total_sales         NUMERIC(14, 2),
    net_profit          NUMERIC(14, 2)
);
```

```sql
-- Tabel C: Tabel Total Grand Makro
CREATE TABLE dm.summary_total_grand (
    total_transaksi_keseluruhan   INT,
    total_customer_keseluruhan    INT
);
```

```sql
-- Tabel B: Tabel Matriks Utama (Brief 2, Brief 3, + Datamart Bebas)
CREATE TABLE dm.matrix_sales_by_store_and_category (
    store_id            INT,
    store_city          TEXT,
    product_id          INT,
    product_name        TEXT,
    category_name       TEXT,
    total_quantity      INT,
    total_sales         NUMERIC(14, 2),
    net_profit          NUMERIC(14, 2),
    total_customer      INT
);
```

Stored Procedure Automation

```sql
-- UPDATE STORED PROCEDURE UNTUK MENGISI 3 TABEL BARU INI
CREATE OR REPLACE PROCEDURE dm.sp_load_matrix_sales()
LANGUAGE plpgsql
AS $$
BEGIN
```

```sql
-- Bersihkan isi data di ketiga tabel sebelum diisi ulang
TRUNCATE TABLE dm.summary_hourly_metrics;
TRUNCATE TABLE dm.matrix_sales_by_store_and_category;
TRUNCATE TABLE dm.summary_total_grand;
```

Insert data dari fact_table

```sql
END;
$$;

CALL dm.sp_load_matrix_sales();
```

Dashboard Preview

Executive Insights & Data Findings

01

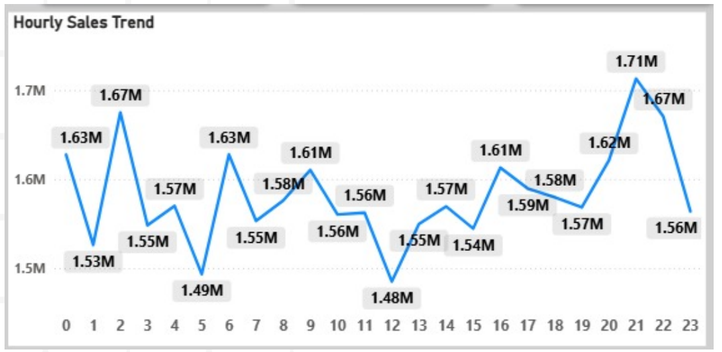
Macro Business Metrics

Total revenue perusahaan berada di angka Rp 38.00M yang diperoleh dari 59K total transactions dan 45K Total customers.

02

Hourly Sales Trend

Peak Hour utama terjadi pada jam 21.00 (9 PM) dengan rekor penjualan tertinggi mencapai Rp 1.71 Million, disusul jam 02.00 pagi dini hari (Rp 1.67 Million). Sementara untuk traffic kuantitas customer, lonjakan volume transaksi terpadat berada pada jam 09.00 pagi (menyentuh kisaran ~2.57K transaksi).



03

Regional Revenue Contribution

Market share antar wilayah cenderung kompetitif dan berimbang merata di kisaran 16% per kota

04

Product Category Performance

Berdasarkan analisis Scatter Plot, kategori Confections dan Meat bertindak sebagai revenue drivers utama di kuadran kanan atas (High Sales, High Qty). Sementara untuk spesifik item, "Eggplant - Asian" memimpin dari segi pendapatan

Strategic Recommendations & Action Plan

Direct Marketing Campaign for Top Drivers

Memfokuskan alokasi marketing budget, program diskon, atau strategi cross-selling pada kategori Confections dan Meat yang terbukti menjadi revenue drivers utama perusahaan dengan volume penjualan dan total sales tertinggi di kuadran Scatter Plot.

Aggressive Regional Expansion

Mengingat performa pendapatan di seluruh cabang saat ini sudah sangat stabil dan seimbang (rata-rata Rp 6.2M - Rp 6.4M per kota), perusahaan sudah siap untuk melakukan ekspansi pasar ke luar daerah dengan menargetkan kota-kota besar baru yang memiliki karakteristik demografi serupa.

Hourly Shift Optimization

Mengoptimalkan alokasi staff kasir aktif menjelang traffic spikes terpadat pada jam 09.00 pagi (puncak kuantitas transaksi), serta mengamankan inventory dan kesiapan restock menjelang jam 21.00 malam yang menjadi puncak nominal sales tertinggi (Rp 1.71 Million).

Driving Retention & Acquisition

Rasio transaksi per pelanggan yang rendah (~1.3x) mengindikasikan lemahnya repeat order. Perusahaan wajib meluncurkan Customer Loyalty Program (seperti sistem poin/member) untuk mendatangkan pelanggan baru sekaligus merangsang pelanggan lama agar kembali berbelanja lebih sering.

Thank You
For Your Attention

Feel free to reach out for any questions or discussion

muhraflyalviansyah@gmail.com

www.linkedin.com/in/raflyalvish/